

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
```

```
df = pd.read_csv('/content/drive/MyDrive/AI ML Bootcamp/Week 1/product_reviews.csv')
df.head()
```



	Review	Sentiment
0	This product exceeded my expectations! It's hi...	Positive
1	The product was decent. It worked fine, but it...	Neutral
2	I had a terrible experience with this company....	Negative
3	It's an okay product. Nothing to write home ab...	Neutral
4	Disappointed with the product. It didn't meet...	Negative

```
df['new_col'] = df['Sentiment'].apply(lambda x: 2 if x == 'Positive' else (1 if x == 'Neutral' else 0))
```

```
df.head()
```



	Review	Sentiment	new_col
0	This product exceeded my expectations! It's hi...	Positive	2
1	The product was decent. It worked fine, but it...	Neutral	1
2	I had a terrible experience with this company....	Negative	0
3	It's an okay product. Nothing to write home ab...	Neutral	1
4	Disappointed with the product. It didn't meet...	Negative	0

```
df['Sentiment'].value_counts()
```



	count
Sentiment	
Positive	129
Negative	129
Neutral	128

```
mapping = {
    'Positive': 1,
    'Negative': 2,
    'Neutral': 0
}
df['Predictor'] = df['Sentiment'].map(mapping)
```

```
df.dtypes
```



	0
Review	object
Sentiment	object
new_col	int64
Predictor	int64

```
X_train, X_test, y_train, y_test = train_test_split(df['Review'], df['new_col'], test_size=0.30)
```

```
cv = CountVectorizer()
X_train_count = cv.fit_transform(X_train.values)
```

```
model = MultinomialNB()
model.fit(X_train_count, y_train)
```

  MultinomialNB ⓘ ?

MultinomialNB()

```
email_not_spam = ["Hello, bro!"]
email_not_spam_count = cv.transform(email_not_spam)
model.predict(email_not_spam_count)
```

 array([0])

```
email_spam = ["Great opportunity to win prize!"]
email_spam_count = cv.transform(email_spam)
model.predict(email_spam_count)
```

 array([2])

```
positive=['The product is so good. I will order it next time']
negative=['The product is worse. it small dirty and packed poorly']
neutral=['The product is average']
```

```
print(model.predict(cv.transform(positive)))
print(model.predict(cv.transform(negative)))
print(model.predict(cv.transform(neutral)))
```

 [2]
[0]
[1]

```
X_test_count = cv.transform(X_test)
model.score(X_test_count, y_test)
y_pred = model.predict(X_test_count)
accuracy = (y_pred == y_test).mean()
print(accuracy)
precision = precision_score(y_test, y_pred, average='weighted')
print(precision)
recall = recall_score(y_test, y_pred, average='weighted')
print(recall)
f1 = f1_score(y_test, y_pred, average='weighted')
print(f1)
```

 0.9913793103448276
0.9915667166416792
0.9913793103448276
0.991362343266615

Start coding or [generate](#) with AI.

